

Informe de Auditoría

Auditoría del sistema web de CEDICA: diagnóstico y recomendaciones

Índice

Índice.....	1
Contexto de la Organización y Sistema a Evaluar.....	2
Riesgo de la Auditoría.....	3
Objetivo de la Auditoría.....	4
Muestreo Utilizado para la Evaluación de la Auditoría.....	4
Resultados de la Evaluación.....	5
Modularidad del Código.....	5
Criterio de Evaluación.....	5
Método de Cálculo de la Métrica.....	5
Resultado de la Evaluación.....	6
Documentación.....	6
Criterio de la Evaluación.....	6
Método de Cálculo de la Métrica.....	7
Resultado de la Evaluación.....	7
Gestión de Logs.....	7
Criterio de Evaluación.....	7
Método de Cálculo de la Métrica.....	8
Resultado de la Evaluación.....	8
Manejo de Excepciones.....	8
Criterio de la Evaluación.....	8
Método de Cálculo de la Métrica.....	9
Resultado de la Evaluación.....	10
Pruebas Automatizadas.....	10
Criterio de la Evaluación.....	10
Método de Cálculo de la Métrica.....	11
Resultado de la Evaluación.....	12
Desacoplamiento entre Componentes.....	12
Criterio de la Evaluación.....	12
Método de Cálculo de la Métrica.....	12
Resultado de la Evaluación.....	13
Estandarización del Código.....	14
Criterio de la Evaluación.....	14
Método de Cálculo de la Métrica.....	14
Resultado de la Evaluación.....	14

Control de Versiones y Trazabilidad de Cambios.....	14
Criterio de la Evaluación.....	14
Método de Cálculo de la Métrica.....	15
Resultado de la Evaluación.....	15
Opinión de Auditoría y Hallazgos.....	16
Plan de mejora y recomendaciones.....	16
Conclusión de la Auditoría.....	18

Contexto de la Organización y Sistema a Evaluar

CEDICA es una Asociación Civil sin Fines de Lucro, fundada en 1994 en la ciudad de La Plata, Provincia de Buenos Aires, Argentina. CEDICA trabaja con la finalidad de IGUALAR OPORTUNIDADES en procura de la reinserción familiar, el logro de la escolaridad, la integración laboral y la mejora integral de la calidad de vida de los J&A (“Jinetes” y “Amazonas”) que forman parte de la institución.

El sistema está compuesto por dos componentes principales:

- Aplicación web interna de administración (para usuarios y administradores) desarrollada en Python con el framework Flask.
 - Portal web institucional desarrollado en Vue.js donde se podrán visualizar los servicios ofrecidos por la institución.
-

Riesgo de la Auditoría

La **evaluación del riesgo de la Auditoría** se realizó siguiendo el modelo estándar de auditoría informática: $RDA = RI * RC * RD$

Donde:

- RDA: Riesgo Deseado de Auditoría.
- RI: Riesgo Inherente.
- RC: Riesgo de Control.
- RD: Riesgo de Detección.

Establecimos el valor del **RDA** en 0.10, es decir, 10%. Para asignar este valor nos basamos en el impacto social del sistema para la comunidad de CEDICA y la necesidad de mantener la confianza institucional que provee.

Luego, en base al análisis del sistema y el contexto organizacional, **se determinaron los siguientes valores:**

Riesgo	Valor Asociado	Justificación
Riesgo Inherente (RI)	0.3	El sistema maneja información sensible de beneficiarios y personal interno de la organización, pero no se considera de alta complejidad tecnológica ni con alta probabilidad de fraude.
Riesgo de Control (RC)	0.6	Existen controles básicos implementados, pero algunos procedimientos de validación o monitoreo no se encuentran completamente formalizados o automatizados, lo que es riesgoso.
Riesgo de Detección (RD)	$RD = RDA/(RI * RC) \approx 0.56$	Este valor de RD indica que nuestros procedimientos de auditoría deben ser lo suficientemente rigurosos como mantener la probabilidad de error por debajo del 56%.

Si bien CEDICA opera en un entorno de baja complejidad técnica y con riesgo inherente moderado, la naturaleza social de su misión exige mantener un estándar adecuado de control y trazabilidad. El análisis de riesgos presentado establece una **base objetiva para definir el alcance y la profundidad de los procedimientos de auditoría**, asegurando que se mantenga un nivel aceptable de confianza en el sistema auditado.

Objetivo de la Auditoría

El objetivo de la presente auditoría es evaluar el cumplimiento de estándares de calidad en el desarrollo de la aplicación web del Centro de Equitación para Personas con Discapacidad y Carenciadas (CEDICA). La auditoría tiene como propósito identificar fortalezas y debilidades en la implementación actual del sistema, y ofrecer recomendaciones concretas para su mejora continua.

La evaluación se realizará en base al análisis de las métricas de Calidad de Software definidas para las siguientes directivas:

- Modularidad del Código.
- Documentación.
- Gestión de Logs.
- Manejo de Excepciones.
- Pruebas Automatizadas.
- Desacoplamiento entre Componentes.
- Estandarización del Código.
- Control de Versiones y Trazabilidad de Cambios.

Dado el análisis del riesgo de la Auditoría, el contexto de uso del sistema y el enfoque adoptado durante el proceso de desarrollo utilizado donde se llevó a cabo una metodología de Scrum con uso de herramientas como Pivotal Tracker, se ha decidido realizar una auditoría con un nivel de rigor bajo. Esta decisión se fundamenta en que el sistema evaluado presenta un impacto crítico reducido, es decir, que una baja calidad en los requisitos evaluados no genera pérdidas económicas directas o riesgos significativos para los usuarios, aunque sí podría afectar la mantenibilidad y escalabilidad futura del sistema.

Muestreo Utilizado para la Evaluación de la Auditoría

Para esta auditoría se decidió trabajar con una muestra compuesta por todos los componentes relacionados con dos módulos funcionales clave de la Aplicación web interna de administración:

1. Módulo de Equipo

- Este módulo gestiona la administración del personal que forma parte de CEDICA. Incluye funcionalidades tales como: alta, modificación y baja de empleados, acceso según roles y visualización estructurada de la información del equipo.

2. Módulo de Jinetes y Amazonas (J&A)

- Este módulo permite la gestión integral de los beneficiarios del sistema. Entre sus funcionalidades se encuentran:
 - Carga de nuevos legajos y documentación.
 - CRUD completo (crear, leer, actualizar, eliminar) de registros de J&A.
 - Paginación de listados y filtros por nombre y apellido.
 - Acceso segmentado según roles (por ejemplo, usuarios técnicos).

Ambos módulos fueron seleccionados por su nivel de criticidad funcional dentro de la aplicación interna y por la diversidad de funcionalidades que encapsulan, permitiendo así un análisis representativo de la calidad modular del sistema en su conjunto.

Resultados de la Evaluación

Modularidad del Código

Criterio de Evaluación

La modularidad del código fuente fue evaluada clasificando los componentes analizados en tres niveles de acuerdo a la cantidad y tipo de funcionalidades que encapsulan:

- **Altamente Modularizado:** Módulo que implementa una única funcionalidad principal, sin dependencias externas
 - Puntaje asignado: 1.0 puntos.
- **Medianamente Modularizado:** Módulo que contiene entre 2 y 3 funcionalidades estrechamente relacionadas.
 - Puntaje asignado: 0.7 puntos.
- **No Modularizado:** Módulo que concentra 4 o más funcionalidades diferentes, o con bajo nivel de cohesión.
 - Puntaje asignado: 0.4 puntos.

Método de Cálculo de la Métrica

La puntuación total obtenida por todos los módulos identificados se divide por la cantidad total de módulos analizados, según la siguiente **fórmula**:

$$\text{Modularidad} = (\text{Suma de puntajes obtenidos por módulo}) / (\text{Cantidad total de módulos analizados})$$

Este valor se interpreta bajo la siguiente **escala**:

Valor de la Métrica	Nivel de Cumplimiento
Mayor o igual a 0.90	Excede los requerimientos
Mayor o igual a 0.85	Rango aceptado
Mayor o igual 0.75	Mínimamente aceptable
Menor a 0.75	Inaceptable

Resultado de la Evaluación

Durante la auditoría se identificaron 74 módulos funcionales distribuidos en tres categorías, según el criterio definido:

- **Altamente Modularizados:** 25 módulos.
- **Medianamente Modularizados:** 37 módulos.
- **No Modularizados:** 12 módulos.

Se aplicó la fórmula definida:

$$\text{Modularidad} = (25 \times 1.0) + (37 \times 0.7) + (12 \times 0.4) / 74 \approx 0.752$$

Resultado de la métrica: 0.752. De acuerdo con la escala establecida, este resultado se encuentra en el nivel **“Mínimamente aceptable”**, ya que es mayor al umbral mínimo aceptable (0.75).

Documentación

Criterio de la Evaluación

La documentación disponible del sistema, si bien fue realizada por un equipo diferente al de desarrollo lo cual podría significar un riesgo respecto a la fidelidad de la documentación frente al sistema implementado, sin embargo, dicho riesgo fue mitigado mediante una verificación detallada de la documentación contra el comportamiento real del sistema, comprobando que las funcionalidades están completamente y correctamente documentadas.

Método de Cálculo de la Métrica

Indicador	Análisis	Puntaje
Compleitud	La documentación cubre todos los aspectos necesarios del sistema	1.0
Exactitud	La información reflejada es correcta y está alineada con el software	1.0
Claridad	Es buena claridad, pero podría mejorarse con diagramas de flujo, de BD, etc para su mayor comprensión	0.8

Cálculo **indicador** **de** **calidad:**

$(completitud * 0.4) + (exactitud * 0.4) + (claridad * 0.2)$

A partir del resultado final, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
Mayor o igual a 0.90	Excede los requerimientos
Mayor o igual a 0.85	Rango aceptado
Mayor o igual 0.75	Mínimamente aceptable
Menor a 0.75	Inaceptable

Resultado de la Evaluación

Al realizar la Evaluación para medir la **Documentación del sistema** tenemos:

$$(1 * 0.4) + (1 * 0.4) + (0.8 * 0.2) = 0.96.$$

Lo que nos da que **Excede los requerimientos.**

Gestión de Logs

Criterio de Evaluación

Actualmente el sistema carece completamente de cualquier tipo de registro (logs), lo que implica un riesgo significativo en términos de:

- Trazabilidad de operaciones
- Diagnóstico ante fallos o bugs en producción
- Auditoría de actividades de usuario

Criterio	Análisis	Puntaje
Timestamp	no aplica	0.25
Formato estructurado	no aplica	0.25
Niveles de severidad	no existe	0.25
Información del evento	no aplica	0.25
Identificación del componente	no aplica	0.25
Configuración de almacenamiento	no existe	0.25

Método de Cálculo de la Métrica

Cálculo **indicador** **de** **calidad:**
cantidad de puntos totales / cantidad de criterios analizados

A partir del resultado final, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
0.8 - 1.0	Satisfactorio
0.4 - 0.8	Aceptable
0.0 - 0.4	Insatisfactorio

Resultado de la Evaluación

Al realizar la Evaluación para medir la **Gestión de Logs del sistema** tenemos:
 $1.5 / 6 = 0.25$.

Lo que nos da es que es **Insatisfactorio.**

Manejo de Excepciones

Criterio de la Evaluación

Se espera que el sistema maneje adecuadamente las excepciones a nivel de aplicación, evitando interrupciones abruptas del servicio y brindando respuestas claras al usuario. Esto implica capturar errores esperables y no esperables, registrar adecuadamente la información asociada y retornar mensajes comprensibles y no técnicos en la interfaz de usuario. Esta dimensión se alinea con la sub característica de *Tolerancia a Fallos* definida en el modelo ISO/IEC 25010.

Criterio	Análisis	Puntaje
Uso de estructuras de captura (try/catch) en puntos críticos del sistema	no aplica	0.25
Registro de excepciones en logs estructurados (con mensaje, tipo y stack trace)	no aplica	0.25

Mensajes de error amigables para el usuario final	no existe	0.25
Comportamiento estable ante fallos (el sistema no se cae)	no aplica	0.25

Método de Cálculo de la Métrica

Criterio	Métrica	Inaceptable	Aceptable	Rango Objetivo	Excede Expectativas
Captura de errores en bloques críticos	% de funciones críticas con try/catch	< 40%	40–69%	70–89%	≥ 90%
Registro estructurado en logs	% de excepciones logueadas con contexto	< 50%	50–69%	70–85%	≥ 86%
Estabilidad del sistema ante errores	N° de caídas por errores no controlados	> 5 en pruebas	3–5 en pruebas	1–2 en pruebas	0
Mensajes de error amigables al usuario	% de errores que devuelven mensaje claro y útil	< 50%	50–74%	75–89%	≥ 90%

cantidad de puntos totales / cantidad de criterios analizados

A partir del resultado final, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
0.8 - 1	Satisfactorio
0.4 - 0.8	Aceptable
0.0 - 0.4	Insatisfactorio

Resultado de la Evaluación

Resultado: 0.0 (try/catch) + 0 (logs) + 0.8 (mensajes) + 1 (estabilidad) = **1.8**

Al realizar la Evaluación para medir la **Manejo de Excepciones** tenemos:
 $1.8 / 4 = 0.45$.

Lo que nos da es que es **Aceptable**.

Pruebas Automatizadas

Criterio de la Evaluación

La auditoría evaluó la existencia y calidad de pruebas automatizadas en los niveles unitario, de integración y funcional. Se espera que el sistema incluya una estrategia de testing que permita validar componentes individuales y su interacción, y que estas pruebas estén integradas en el flujo de desarrollo.

Criterio Evaluado	Puntaje
Cobertura de código en pruebas unitarias	0.25
Integración en proceso CI/CD (automatización)	0.25
Existencia de pruebas funcionales sobre flujos clave	0.25
Generación de reportes de pruebas	0.25
Detección de regresiones	0.25

Método de Cálculo de la Métrica

Criterio	Métrica	Inaceptable	Aceptable	Rango Objetivo	Excede Expectativas
Cobertura de código (unitarias)	% de líneas de código cubiertas	< 30%	30–59%	60–85%	≥ 85%

		por pruebas unitarias				
Cobertura de flujos críticos (funcionales)		% de funcionalidades principales cubiertas por pruebas E2E	< 50%	50–74%	75–89%	≥ 90%
Integración CI/CD	en	¿Se ejecutan automáticamente en cada commit o merge?	>No	Parcial (manual)	Sí, con fallos ocasionales	Sí, estable y con reportes
Detección regresiones	de	N° de regresiones no detectadas antes del despliegue	> 5	3–5	1–2	0
Reportes trazabilidad resultados	y de	¿Se generan reportes automatizados con resultados de pruebas?	No	Sí, pero incompletos	Sí, con métricas básicas	Sí, detallados y vinculados a tickets/issues

cantidad de puntos totales / cantidad de criterios analizados

A partir del resultado final, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
---------------------	-----------------------

0.8 - 1	Satisfactorio
0.4 - 0.8	Aceptable
0.0 - 0.4	Insatisfactorio

Resultado de la Evaluación

Resultado: 0 (unitarias) + 0 (funcionales) + 0 (CI/CD) + 0 (reportes) + 0 (regresiones) = 0.0

Al realizar la Evaluación para medir las **Pruebas automatizadas** tenemos: 0 / 5 = 0.

Lo que nos da es que es **Insatisfactorio**.

Desacoplamiento entre Componentes

Criterio de la Evaluación

Para medir el desacoplamiento entre los componentes, hay que tener en cuenta los siguientes criterios para cada componente:

- **Fan-in:** Cuántos componentes dependen de este componente.
- **Fan-out:** Cuántos componentes este componente depende.

Método de Cálculo de la Métrica

La **escala de cada categoría** es:

- **Fan-in:** *Componentes dependientes del componente / (Componentes totales – 1).*
- **Fan-out:** *Componentes que el componente depende / (Componentes totales – 1).*

El **nivel de desacoplamiento de un componente** es $1 - (Fan\ in + Fan\ out) / 2$.

El **nivel de desacoplamiento del producto** es $(\sum Nivel\ de\ desacoplamiento\ del\ componente) / Componentes\ totales$.

A partir del resultado final, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
Mayor o igual a 0.85	Excede los requerimientos
Mayor o igual a 0.75	Rango aceptado
Mayor o igual 0.65	Mínimamente aceptable

Menor a 0.65

Inaceptable

Resultado de la Evaluación

Componente de servicios del Equipo:

- Fan-in: $2/3 = 0,67$
- Fan-out: $0/3 = 0$
- Nivel de desacoplamiento: $1 - (0,67 + 0)/2 = 0,67$

Componente de controlador del Equipo:

- Fan-in: $0/3 = 0$
- Fan-out: $1/3 = 0,33$
- Nivel de desacoplamiento: $1 - (0 + 0,33)/2 = 0,84$

Componente de servicios de J&A:

- Fan-in: $1/3 = 0,33$
- Fan-out: $0/3 = 0$
- Nivel de desacoplamiento: $1 - (0,33 + 0)/2 = 0,84$

Componente de controlador de J&A:

- Fan-in: $0/3 = 0$
- Fan-out: $2/3 = 0,67$
- Nivel de desacoplamiento: $1 - (0 + 0,67)/2 = 0,67$

Nivel de desacoplamiento total: $(0,67 + 0,84 + 0,84 + 0,67)/4 = 0,76$

El **nivel de desacoplamiento** está en el rango **aceptado**.

Estandarización del Código

Criterio de la Evaluación

En este caso, las líneas de código fueron escritas en Python, por lo que se va a analizar el código en base al documento PEP8 de Python, que determina las guías de estilo del lenguaje. En base a esto, se va a buscar la cantidad de líneas de código entregables que presenten violaciones de la guía.

Método de Cálculo de la Métrica

La forma de medir la **estandarización de código** será:
 $1 - (\text{Líneas con violaciones} / \text{total de líneas de código entregables})$.

A partir del resultado de esa fórmula, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
Mayor o igual a 1.0	Excede los requerimientos
Mayor o igual a 0.9	Rango aceptado
Mayor o igual 0.8	Mínimamente aceptable
Menor a 0.8	Inaceptable

Resultado de la Evaluación

En los módulos analizados se encontraron 361 líneas con violaciones, siendo el total de líneas 2235. La fórmula nos da:

$$1 - (361/2235) = 0,84$$

Según la métrica, la estandarización es **mínimamente aceptable**.

Control de Versiones y Trazabilidad de Cambios

Criterio de la Evaluación

Evaluamos el uso del sistema de control de versiones (Git) en el desarrollo del sistema web interno de CEDICA analizando los commits en los repositorios relacionados con los módulos de Equipo y Jinetes y Amazonas.

Se usaron los siguientes criterios por commit:

Criterio	Descripción
Claridad del mensaje	¿El mensaje explica con claridad qué se hizo?
Referencia a tarea o issue	¿Menciona un ID o título de una tarea (por ejemplo, de Pivotal Tracker)?
Frecuencia adecuada	¿Se realizan commits con una frecuencia razonable (no demasiado espaciados ni acumulados)?
Tamaño adecuado	¿Los commits abordan cambios manejables, evitando "mega commits"?

Se evaluaron cada uno de los commits con los siguientes puntajes:

- **Aceptable (1.0 punto):** Cumple con todos los criterios.
- **Intermedio (0.5 punto):** Cumple parcialmente (por ejemplo, mensaje claro pero sin referencia a issue).
- **Inadecuado (0.25 punto):** Incumple varios criterios (mensaje confuso, sin referencia, muy grande, frecuencia mala).

Método de Cálculo de la Métrica

Finalmente se aplica la **fórmula**:

$$\text{Puntaje Total} = (\text{Suma de puntajes de commits analizados}) / (\text{Cantidad total de commits analizados})$$

A partir del resultado de esa fórmula, establecemos los valores para la **escala de la métrica**:

Valor de la Métrica	Nivel de Cumplimiento
Mayor o igual a 0.90	Excede los requerimientos
Mayor o igual a 0.75	Rango aceptado
Mayor o igual 0.60	Mínimamente aceptable
Menor a 0.60	Inaceptable

Resultado de la Evaluación

Resultados Obtenidos

- Total commits evaluados: 30
- Suma de puntajes: 23.25
- Puntaje total promedio: $23.25 / 30 = 0.775$

Resultado obtenido: 0.775 → Cumple dentro del **rango aceptado**.

Opinión de Auditoría y Hallazgos

De la presente auditoría se pudieron observar los siguientes **puntos a tener en cuenta** como **hallazgos relevantes** para una próxima mejora del sistema web de CEDICA:

- Un **33.78%** de los módulos están altamente modularizados, lo cual es positivo. Sin embargo, un **16.22%** de los módulos no respetan criterios adecuados de modularización, lo que puede afectar la mantenibilidad y la escalabilidad del sistema.

- Se observa que la **documentación** tiene una cobertura completa de las funcionalidades y una alineación correcta con el comportamiento del sistema aunque la **claridad de la información podría mejorarse**, especialmente en lo visual. La falta de diagramas o modelos gráficos puede dificultar la comprensión rápida del sistema por parte de nuevos desarrolladores o mantenedores.
 - Actualmente **no se implementa ningún mecanismo de logging**, lo que genera una carencia importante en la capacidad de **diagnóstico ante fallos, auditoría de usuarios y trazabilidad operativa**.
 - **No se utilizan bloques try/except** en partes críticas del código.
 - **No existe una política estandarizada para el registro de errores en logs**, lo cual se relaciona también con la falta de implementación de logging estructurado.
 - Los **mensajes de error** son entendibles para el usuario no técnico.
 - **No se detectaron caídas del sistema ante errores simulados**.
 - **No existen pruebas unitarias** para modelos y algunos controladores.
 - **No se encontraron pruebas funcionales automatizadas**.
 - El sistema **no cuenta con un pipeline CI/CD** configurado para ejecutar pruebas automáticamente.
 - **No se generan reportes automáticos de resultados de pruebas**.
 - Existe un leve acoplamiento entre los componentes, pero lo suficiente para **no considerarse una pérdida**.
 - Se hallaron **más de 300 líneas** que no respetan la PEP 8 de Python, especialmente en cuanto a la **longitud de las líneas** (superan los 79 caracteres de longitud).
 - Algunos **commits** encontrados **no fueron descriptivos o utilizaban nombres informales**, lo que dificulta la trazabilidad.
-

Plan de mejora y recomendaciones

Con base en los hallazgos detectados durante la auditoría, se propone el siguiente **plan de mejora** con el **objetivo** de guiar a CEDICA en la mejora continua de su sistema, priorizando aquellos aspectos que representan un mayor riesgo operativo:

1. **Revisión de módulos no modularizados:** Identificar los 12 módulos con 4 o más funcionalidades, dividir sus responsabilidades y aplicar principios como Single Responsibility Principle (SRP).
2. **Refactorización progresiva:** Priorizar la modularización en módulos con mayor tasa de cambios o mayor importancia funcional.
3. **Establecer políticas de revisión de código:** Incorporar revisiones que verifiquen el nivel de modularización antes de hacer merge de funcionalidades en ramas principales.
4. **Complementar la documentación con diagramas** de componentes, arquitectura lógica y flujos de datos para facilitar la mantenibilidad del sistema.
5. En caso de que el sistema evolucione, es importante **definir una política de actualización documental** con cada nueva release.

6. **Implementar el sistema de logging:** Para un sistema sin logs, es crucial incorporar una librería de logging adecuada (como “logging” en Python), definir un formato estandarizado para los mensajes (con timestamp, nivel de severidad, mensaje, componente y usuario), configurar el almacenamiento y la rotación de archivos, y documentar estos mecanismos para asegurar la trazabilidad y facilitar el diagnóstico y la auditoría del sistema.
7. **Definir un formato estandarizado** con los siguientes campos:
 - Timestamp (con zona horaria)
 - Nivel de severidad (INFO, DEBUG, ERROR, etc.)
 - Mensaje descriptivo
 - Componente afectado
 - Identificador de usuario o sesión
8. **Configurar almacenamiento con rotación de archivos** o exportación hacia un sistema de recolección.
9. **Documentar estos mecanismos en el manual técnico**
10. **Implementar bloques try-except:** Asegurar que todas las operaciones que puedan fallar (por ejemplo, interacción con bases de datos, llamadas a servicios externos, validación de entrada de usuario) estén envueltas en bloques try/except.
11. **La ausencia total de pruebas automatizadas requiere implementar un plan integral:** se debe seleccionar un “framework” de pruebas adecuado, priorizar el desarrollo de pruebas unitarias para la lógica de negocio crítica, continuar con pruebas de integración para la interacción entre componentes, y considerar pruebas funcionales o de extremo a extremo para flujos clave; además, es fundamental integrar la ejecución automática de estas pruebas en un sistema de Integración Continua (CI) para detectar regresiones tempranamente, establecer métricas de cobertura de código y fomentar una cultura de desarrollo guiado por pruebas en el equipo.
12. Implementar una **convención clara para mensajes de commit** (por ejemplo: tipo de cambio + módulo + referencia).
13. **Usar siempre un ID de tarea o issue** en cada commit.
14. **Promover el uso de commits más pequeños y frecuentes** para facilitar la revisión de cambios.

A su vez, dejamos declaradas ciertas **recomendaciones que consideramos que deben ser tomadas a un Corto Plazo** para mejorar los aspectos **más críticos** del sistema según lo hallado durante la auditoría:

- **Implementación básica de login estructurado** incorporando el uso de una librería o herramienta especializada para la tarea, en pos de asegurar la trazabilidad de acciones claves del sistema.
 - **Desarrollo e integración de pruebas automatizadas unitarias** para los modelos y controladores principales del sistema, definiendo a su vez casos de prueba para funcionalidades críticas del sistema.
 - **Establecimiento de lineamientos mínimos de calidad de código y revisiones obligatorias del mismo.**
-

Conclusión de la Auditoría

De acuerdo con los objetivos planteados en esta auditoría, se realizó una evaluación de la calidad de la aplicación web de CEDICA en base a ocho directivas técnicas clave. La auditoría se desarrolló bajo un enfoque de mejora continua, considerando un **Riesgo Deseado de Auditoría (RDA) de 10%**, que representa el nivel máximo de riesgo aceptable para la organización.

A partir del análisis, se identificaron las siguientes situaciones:

- **Fortalezas:** Se destaca un cumplimiento satisfactorio o superior a lo requerido en aspectos como la **documentación**, el **manejo de excepciones**, y la **trazabilidad de cambios**.
- **Debilidades:** Se detectaron deficiencias en la **gestión de logs** y en las **pruebas automatizadas**, que deberán ser abordadas a corto o mediano plazo para evitar afectar la mantenibilidad del sistema.

Teniendo en cuenta el **contexto de uso del sistema**, el impacto limitado de las debilidades encontradas, y el hecho de que **el riesgo de auditoría permanece dentro de los márgenes aceptables**, se concluye que:

La auditoría resultó exitosa en tanto que logró verificar el cumplimiento básico de los estándares de calidad definidos y ofrecer un conjunto de recomendaciones accionables para fortalecer el sistema en áreas puntuales.